Design and Implementation of a Custom 16-Bit RISC SoC Digital Lock System on Nexys A7-100T FPGA

Justin Natividad
College of Engineering
California State University, Long Beach
Long Beach, CA, USA
justin.natividad01@student.csulb.edu
Matthew Orendain
College of Engineering
California State University, Long Beach
Long Beach, CA, USA
matthew.orendain01@student.csulb.edu
Gabriel Dao
College of Engineering
California State University, Long Beach
Long Beach, CA, USA
gabriel.dao01@student.csulb.edu
Mikko Taitague
College of Engineering
California State University, Long Beach
Long Beach, CA, USA
mikko.taitague01@student.csulb.edu

Abstract*—This paper presents the design and implementation of a custom 16-bit Reduced Instruction Set Computer (RISC) System-on-Chip (SoC) targeting the Digilent Nexys A7-100T FPGA evaluation board. The system features a single-cycle Harvard-architecture CPU core with a 16-instruction custom ISA, an 8-register file (R0–R7, R0 hardwired to zero), a 2-flag ALU (Z and N), a 256-word data RAM, a 1024-word instruction ROM, and a memory-mapped I/O subsystem supporting push buttons, switches, LEDs, an RGB LED, and an 8-digit seven-segment display. The SoC executes a digital combination-lock application stored in ROM: the user navigates and enters a 4-digit hexadecimal passcode (1738) using the onboard directional buttons; correct entry triggers a PASS sequence with green RGB feedback, while three consecutive incorrect entries trigger a LOCKOUT state and halt the processor. The design was first verified in Logisim Evolution and then implemented in Verilog using Xilinx Vivado 2024, targeting the Artix-7 FPGA fabric. Functional simulation testbenches were written for every module and the integrated CPU core, and synthesis was completed successfully with the design checkpoint (DCP) generated.

***Keywords*—RISC, SoC, FPGA, Nexys A7, Harvard Architecture, ISA, Memory-Mapped I/O, Digital Lock, Verilog, LogisimEvolution** INTRODUCTION (*HEADING 1*)

I. INTRODUCTION

The design and verification of a complete processor system from first principles is one of the most comprehensive exercises in digital engineering education. This project, completed as the final deliverable for CECS 361 – Digital Design Techniques and Verification at California State University, Long Beach, challenges students to apply knowledge of sequential and combinational logic, hardware description languages, and FPGA toolchains to build a fully functional computing system.

Our team designed a custom 16-bit RISC SoC that executes a digital combination-lock application. The SoC encompasses a complete processor pipeline: instruction fetch from ROM, instruction decode, register-file access, ALU execution, data-memory access, and write-back. All board peripherals—buttons, switches, LEDs, RGB LED, and seven-segment display—are accessed through a memory-mapped I/O (MMIO) address space visible to the CPU as ordinary LOAD and STORE instructions.

The lock program demonstrates real interaction with the hardware: on power-up the display flashes dashes three times, then enters a ready state showing '----' on the left four digits and the attempt count on the rightmost digit. The user presses CENTER to enter selection mode, navigates four hex digits with LEFT/RIGHT, increments or decrements values with UP/DOWN (0–F with wrap), and confirms with CENTER. A correct passcode (1738) shows 'PASS' blinking with green RGB; three wrong attempts trigger LOCKOUT with flashing segments, followed by CPU HALT.

The remainder of this report is organized as follows. Section II reviews background concepts and tools. Section III details the full implementation. Section IV evaluates results and verification. Section V concludes with lessons learned.

A. Selecting a Template (Heading 2)

First, confirm that you have the correct template for your paper size. This template has been tailored for output on the A4 paper size. If you are using US letter-sized paper, please close this file and download the Microsoft Word, Letter file.

B. Maintaining the Integrity of the Specifications

II. II. BACKGROUND AND PRELIMINARIES

A. A. RISC Architecture

Reduced Instruction Set Computers (RISC) favor a small, uniform instruction set with fixed-length encodings and load/store memory access. All arithmetic and logic operations work only on registers; memory is accessed exclusively via dedicated LOAD and STORE instructions [1]. This simplifies the control unit and enables single-cycle execution with predictable timing—ideal for FPGA implementations where LUT resources are limited.

B. B. Harvard Architecture

The Harvard architecture separates instruction memory from data memory, allowing simultaneous access in a single clock cycle. This contrasts with Von Neumann, where a shared bus serializes instruction fetch and data access [2]. Our design uses a 1024×16-bit instruction ROM and a separate 256×16-bit data RAM, giving a clean Harvard split with no structural hazard between fetch and memory access.

C. C. Memory-Mapped I/O

Memory-mapped I/O (MMIO) assigns peripheral registers to addresses in the normal memory space. The CPU interacts with peripherals using the same LOAD and STORE instructions used for data memory; a hardware address decoder determines whether an access targets RAM or a

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

peripheral [3]. This simplifies the ISA by avoiding special I/O instructions.

D. D. Tools and Resources

Logisim Evolution (v3.8) was used for schematic-level capture and simulation of the complete SoC before any HDL was written. Xilinx Vivado 2024 was used for Verilog RTL entry, functional simulation, synthesis targeting the XC7A100T-CSG324 Artix-7 FPGA, and implementation. The reference textbook was Pong P. Chu's *FPGA Prototyping by Verilog Examples* [4].

III. III. Implementation

The complete SoC consists of nine Verilog modules organized in a strict hierarchy under a single board-level top module. Fig. 1 illustrates the top-level block diagram.

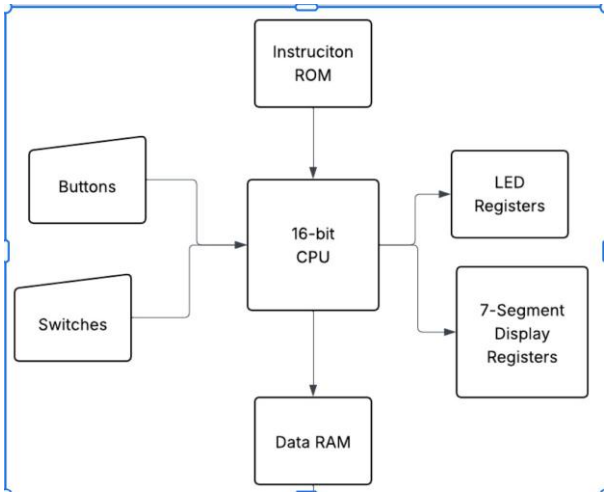

Fig. 1. Fig. 1. Top-level SoC block diagram.

A. A. Top Module (RISC_CPU_top.v)

The board-facing top module **RISC_CPU** instantiates and wires all sub-modules. It accepts the 100 MHz board clock (`CLK100MHZ`), active-low reset (`CPU_RESETN`), 16 switches, and five directional push buttons. It drives 16 LEDs, one RGB LED, and the eight-digit seven-segment display. All asynchronous inputs pass through a two-stage flip-flop synchronizer (`sync_2ff`) before reaching the CPU core, preventing metastability.

B. B. Clock Enable (clock_enable.v)

The **clock_enable** module divides the 100 MHz FPGA clock by a configurable `DIVISOR` parameter (default 50,000, giving a 2 kHz CPU rate). The CPU core executes exactly one instruction per `ce` pulse, making program timing predictable.

C. C. Two-FF Synchronizer (sync_2ff.v)

sync_2ff is a generic, parameterized two-flip-flop synchronizer. The asynchronous input is captured by a first flip-flop (`meta_reg`) that may briefly be metastable, then re-sampled by a second (`sync_reg`) that is guaranteed stable. The WIDTH parameter allows synchronizing 1-bit or 16-bit signals.

D. D. CPU Core (risc_cpu_core.v)

The CPU core implements a single-cycle datapath: every enabled clock edge fetches, decodes, executes, and writes back exactly one instruction. It contains an 8-register file (R0–R7, R0 hardwired to zero), an ALU supporting ADD/SUB/AND/OR/XOR/CMP, Z and N condition flags, and a 16-bit program counter. Table I lists the full 16-instruction ISA.

TABLE I. **TABLE I. Custom 16-Bit ISA Instruction Set**

Opcode	Mnemonic	Encoding	Operation
0x0	`LDI Rd, imm9`	`[0000|Rd|imm9]`	Rd = zero_ext(imm9)
0x1	`LD Rd, addr`	`[0001|Rd|addr[8:0]]`	Rd = MEM[addr] or MMIO
0x2	`ST Rd, addr`	`[0010|Rd|addr[8:0]]`	MEM[addr] = Rd
0x3	`ADD Rd, Rs`	`[0011|Rd|Rs|xxx]`	Rd = Rd + Rs
0x4	`SUB Rd, Rs`	`[0100|Rd|Rs|xxx]`	Rd = Rd - Rs
0x5	`AND Rd, Rs`	`[0101|Rd|Rs|xxx]`	Rd = Rd AND Rs
0x6	`OR Rd, Rs`	`[0110|Rd|Rs|xxx]`	Rd = Rd OR Rs
0x7	`CMP Rd, Rs`	`[0111|Rd|Rs|xxx]`	Flags = Rd - Rs (no WB)
0x8	`JMP addr12`	`[1000|addr12]`	PC = addr12
0x9	`JZ addr12`	`[1001|addr12]`	if Z=1: PC = addr12
0xA	`JNZ addr12`	`[1010|addr12]`	if Z=0: PC = addr12
0xB	`JN addr12`	`[1011|addr12]`	if N=1: PC = addr12
0xC	`NOP`	`[1100|xxxxxxxxxxxx]`	No operation
0xD	`HALT`	`[1101|xxxxxxxxxxxx]`	Halt until reset
0xE	`XOR Rd, Rs`	`[1110|Rd|Rs|xxx]`	Rd = Rd XOR Rs
0xF	`PASS Rd, Rs`	`[1111|Rd|Rs|xxx]`	Rd = Rs

E. E. Instruction ROM (instr_rom.v)

instr_rom implements a 1024×16-bit ROM initialized from `program.mem` via `$readmemh`. All unused locations default to HALT (0xD000). Reads are combinational, supporting single-cycle execution.

F. F. Data RAM (data_ram.v)

data_ram implements a 256×16-bit synchronous read-write memory. Writes are gated by the CPU clock enable and write-enable signal; reads are combinational. On reset, all 256 locations are zeroed.

G. G. Memory-Mapped I/O (mmio.v)

mmio decodes the 9-bit load/store address to determine whether it targets a peripheral (0x034–0x03F). The `hit` output steers LOAD/STORE to MMIO registers instead of RAM. Table II lists the complete address map.

TABLE II. **TABLE II. MMIO Address Map**

Address	Name	Direction	Description
`0x034`	`BT_Reg`	Read-only	Button inputs: {C,U,D,L,R}=bits[4:0]
`0x035`	`SW_Reg`	Read-only	Switch inputs SW[15:0]
`0x036`	`LED_Reg`	Read/Write	LED outputs LED[15:0]
`0x037`	`RGB_Reg`	Read/Write	RGB LED: bits[2:0]={R,G,B}
`0x038`	`Seg7`	Read/Write	Leftmost 7-seg digit
`0x039`	`Seg6`	Read/Write	7-seg digit 6
`0x03A`	`Seg5`	Read/Write	7-seg digit 5

Address	Name	Direction	Description
0x03B	Seg4	Read/Write	7-seg digit 4
0x03C	Seg3	Read/Write	7-seg digit 3
0x03D	Seg2	Read/Write	7-seg digit 2
0x03E	Seg1	Read/Write	7-seg digit 1
0x03F	Seg0	Read/Write	Rightmost 7-seg digit

H. H. Seven-Segment Display

sevenseg_decode converts 16-bit display codes to 7-bit active-high segment patterns. Special codes: 0x0080 (blank), 0x0040 (dash), 0x00FF (all on), 0x0067 ('P'), 0x0047 ('F'). All others display the low nibble as hex 0–F. **sevenseg_scan** multiplexes eight digit registers at 1 kHz, cycling AN[7:0] with the active-low cathode pattern for the selected digit.

I. I. Digital Lock Application (program.mem)

The lock program implements the state machine shown in Fig. 2:

- **STARTUP:** Flash '----' three times on Seg7–Seg4, then enter READY_WAIT.
- **READY_WAIT:** Show '----' on Seg7–Seg4, attempt count on Seg0, poll CENTER.
- **SELECT_DIGIT:** LEFT/RIGHT navigates digits, UP/DOWN changes value (0–F wrap), CENTER submits.
- **CHECK_CODE:** Compare against passcode 1738. Correct → PASS. Wrong → increment fail count; if 3 → LOCKOUT, else READY_WAIT.
- **PASS:** Blink 'PASS' three times with green RGB, then hold.
- **FAIL:** Blink 'FAIL' three times with red RGB, return to READY_WAIT.
- **LOCKOUT:** Flash all segments three times, then HALT.

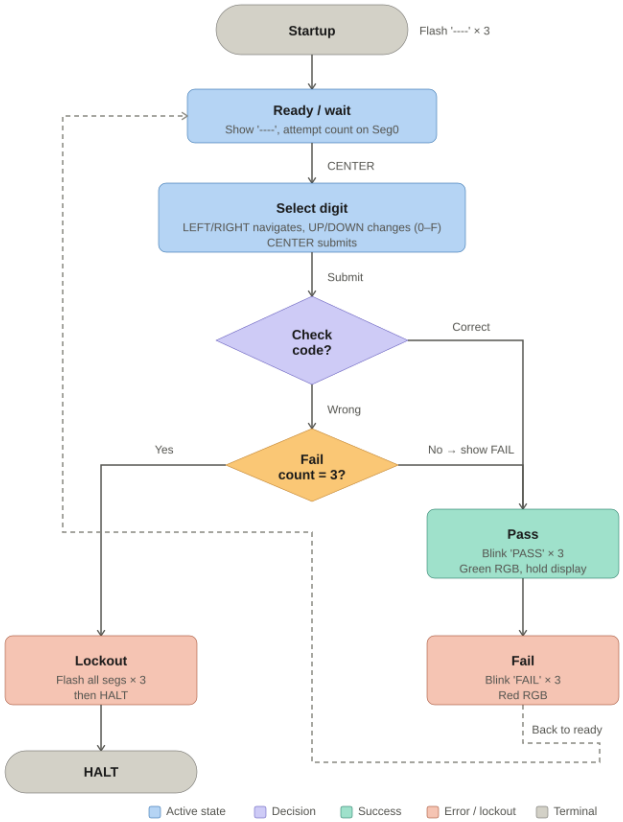

Fig. 2. Fig. 2. Digital lock application state machine.

IV. IV. Evaluation

The project was completed as intended. The full SoC was verified in Logisim Evolution, then re-implemented in Verilog with a comprehensive set of module-level testbenches in Vivado. Synthesis completed successfully, generating a design checkpoint (RISC_CPU.dcp). Due to equipment scheduling constraints, physical board programming was not completed before the submission deadline; all verification results are from simulation.

A. A. Logisim Evolution Verification

The Logisim schematic (`CECS361_RISC_CPU_Design_V3.circ`) implements the complete SoC with the CPU datapath, instruction ROM loaded with program.mem, data RAM, and MMIO blocks. Step-by-step simulation confirmed correct PC sequencing, register-file updates, flag generation, and MMIO reads/writes.

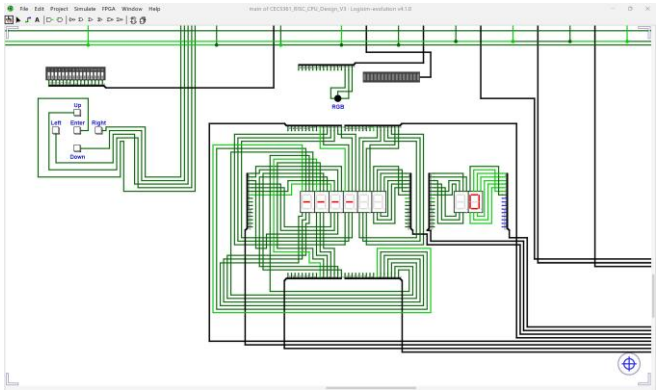

Fig. 3. Fig. 3. Logisim Evolution — CPU executing startup flash sequence.

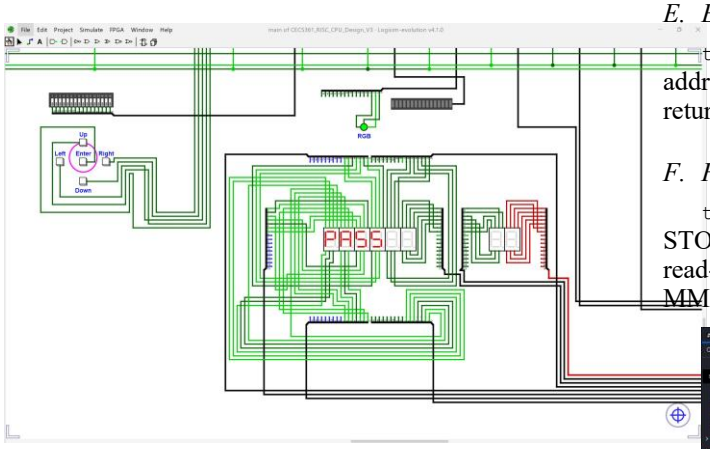

Fig. 4. Fig. 4. Logisim Evolution — PASS state with green RGB output.

B. B. clock_enable Testbench

`tb_clock_enable.v` instantiates `clock_enable` with DIVISOR=4. It verifies that reset forces `ce` low and that `ce` pulses exactly once every four clock cycles. Both conditions passed with zero errors.

Fig. 5. Fig. 5. tb_clock_enable — ce pulses every 4 cycles.

C. C. sync_2ff Testbench

`tb_sync_2ff.v` drives a 1-bit asynchronous input through the two-FF synchronizer and verifies the output is delayed by exactly two clock edges, and confirms WIDTH=8 bus synchronization. All checks passed.

D. D. data_ram Testbench

`tb_data_ram.v` performs write-read round-trip tests on all 256 addresses and verifies synchronous reset zeroes every location. All 256 locations passed.

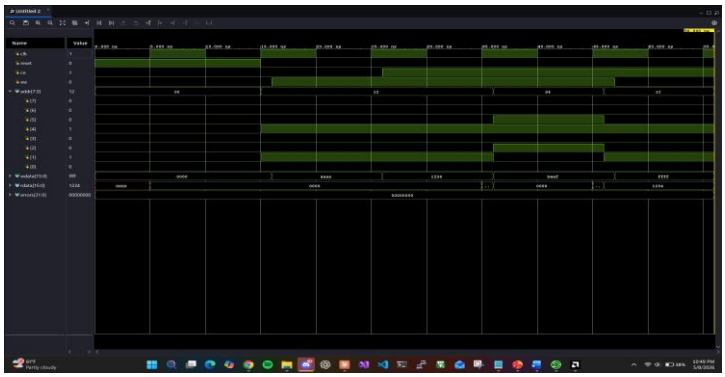

Fig. 6. Fig. 6. tb_data_ram — write/read cycle verification.

E. E. instr_rom Testbench

`tb_instr_rom.v` verifies combinational output across addresses 0–1023 and confirms that out-of-range addresses return HALT (0xD000).

F. F. mmio Testbench

`tb_mmio.v` exercises all 12 MMIO addresses, verifying STORE updates to LED, RGB, and Seg0–Seg7 registers, read-only behavior of BT_Reg and SW_Reg, and that non-MMIO addresses drive `hit=0`.

and seven-segment registers.

`tb_sevenseg_decode.v` applies all five special display codes and all 16 hex digits (0x0000–0x000F). All 21 test vectors passed with zero errors.

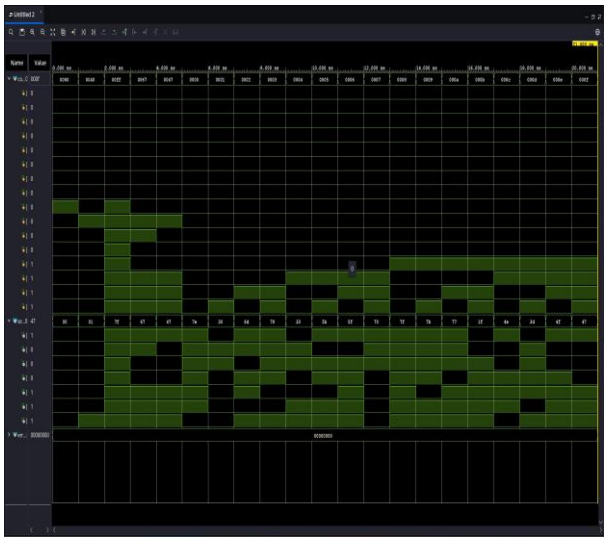

Fig. 8. Fig. 8. tb_sevenseg_decode — hex digit 0–F segment patterns.

H. H. Button Map Testbench

`tb_RISC_CPU_top_button_map.v` verifies the button packing order: bit0=Right, bit1=Down, bit2=Left, bit3=Up, bit4=Center. All six test vectors passed.

I. I. CPU Core Integration Testbench

`tb_risc_cpu_core_startup.v` instantiates the full CPU core with program.mem loaded and verifies the startup sequence (dashes on Seg7–Seg4) and READY_WAIT state. It then simulates CENTER (0x10) and UP (0x08) button presses, verifying digit RAM increments and flag writes.

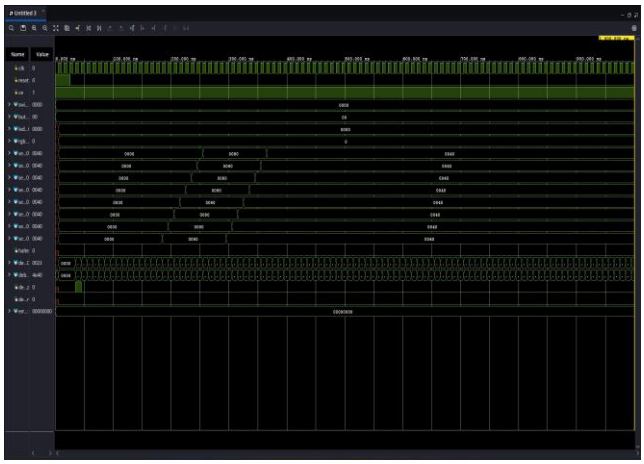

Fig. 9. Fig. 9. tb_risc_cpu_core_startup — startup flash and READY_WAIT state.

Fig. 10. Fig. 10. tb_risc_cpu_core_startup — CENTER press (0x10) and UP press (0x08).

J. J. Synthesis Results

The design was synthesized targeting the XC7A100T-CSG324-1 Artix-7 device. Synthesis completed with no errors. Resource utilization: 2077 LUTs (3%), 4450 slice registers (4%), 58 bonded IOBs (28%). The design checkpoint (`RISC_CPU.dcp`) was generated successfully.

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Bonded IOB (210)	BUFGCTRL (32)
RISC_CPU	2077	4450	567	263	58	1
u_cpu (risc_cpu_core)	2059	4374	567	263	0	0

Fig. 11. Fig. 11. Vivado utilization report — RISC_CPU on xc7a100tcsg324-1.

V. V. Conclusion

This project successfully demonstrated the design and implementation of a complete custom 16-bit RISC SoC from scratch. The team defined a 16-instruction ISA, built a single-cycle Harvard CPU core, connected it to separate instruction ROM and data RAM, and designed a memory-mapped I/O subsystem interfacing all Nexys A7 peripherals. A 4-digit hexadecimal combination lock with startup animation, interactive digit entry, pass/fail feedback, and lockout protection was written in assembly, assembled to hex, and loaded into the instruction ROM.

Verification was conducted at two levels: Logisim Evolution validated the schematic-level SoC, and eight Verilog testbenches verified every module in isolation plus the integrated CPU core. Synthesis completed cleanly in Vivado for the Artix-7 target.

The primary challenge was the complexity of the assembly program: coordinating blink timing loops, button polling debounce, and multi-digit navigation within a 16-instruction ISA required careful use of data RAM for state variables. Future work could include pipelining the CPU, adding a hardware multiplier, VGA output, or UART peripheral—all within the same MMIO framework.

VI. . REFERENCES

[1] [1] D. A. Patterson and J. L. Hennessy, "Computer Organization and Design: The Hardware/Software Interface," 5th ed. Waltham, MA: Morgan Kaufmann, 2014.

[2] [2] A. Bindal, "Fundamentals of Computer Architecture and Design." New York, NY: Springer, 2017.

[3] [3] W. Stallings, "Computer Organization and Architecture," 10th ed. Hoboken, NJ: Pearson, 2016.

[4] [4] P. P. Chu, "FPGA Prototyping by Verilog Examples." Hoboken, NJ: John Wiley & Sons, 2008.

[5] [5] Digilent Inc., "Nexys A7 Reference Manual," 2020. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[6] [6] Xilinx Inc., "Vivado Design Suite User Guide: Logic Simulation," UG900, v2022.1, 2022.

[7] [7] C. Burch, "Logisim Evolution," GitHub, 2023. [Online]. Available: https://github.com/logisim-evolution/logisim-evolution

[8] [8] Digilent Inc., "Nexys A7-100T Master XDC," GitHub, 2018. [Online]. Available: https://github.com/Digilent/digilent-xdc